This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Model Checking Techniques Applied to Satellite Operational Mode Management

Vittoria Nardone, Antonella Santone, Massimo Tipaldi, Davide Liuzza, and Luigi Glielmo

***Abstract*—Satellites are nowadays complex systems and can be considered as components of larger mission-level systems of systems. The increasing complexity of space mission objectives is actually complicating the requirement engineering process. It is generally understood that space system engineers should translate system-level requirements (elaborated in natural language) into verifiable models, which can expose the design issues before the satellite manufacturing phase. This paper shows how the verification of complex system requirements can be performed via model checking. More specifically, a methodology is proposed which exploits the flexibility provided by the calculus of communicating systems to model complex system concurrent parts and their mutual interactions for verifying analytically their correctness, completeness, and consistency as prescribed by the system requirements. The proposed methodology is applied to the verification of a real satellite operational mode management specification. An abstraction reduction technique based on the selective mu-calculus logic is used to address the computational issues in model checking. It allows capturing and analyzing the parts of a satellite involved in the verification of a specific set of its system-level properties.**

***Index Terms*—Abstraction techniques, calculus of communicating systems (CCS), formal verification, model checking, satellite operational mode management (SOMM), systems of systems requirement verification.**

I. Introduction

SPACE mission design involves the integration of several engineering disciplines in order to address the implementation of complex systems requirements and the management of projects over their life-cycle [1]. Space system engineering is an evolving field. Nowadays, it starts to adopt peculiar system of systems engineering methods, practices, and tools [2]. Satellites are complex systems and, regardless of their size [3], can be actually considered as systems of systems (SoS). Their subsystems, such as the electrical power subsystem, interact with all other subsystems on-board for successful mission execution [3]. Satellites can be also regarded as components of a larger mission-level SoS [2]. Space mission requirements are indeed implemented by three synergistic physical systems: the ground segment, the launcher segment, and the space segment. The same remark can be applied to distributed satellite systems, such as satellite constellations and federated satellite systems [4].

The increasing complexity of space mission objectives complicates both the requirement engineering process [2] and their validation [5]. Current testing activities are labor intensive and are performed on flight and flight-like hardware as well as on satellite simulators [6]. Such test environments are usually unavailable during the early phases of space projects. Model checking is a formal verification method and is based on rather abstract behavioral models [7]. Requirements engineers and space mission experts can greatly benefit from using model checking for the elicitation, the analysis, and the verification of space system requirements. Model checking techniques use strict mathematical reasoning to prove that a system conceptual design, modeled as a concurrent finite state machine, fulfills a set of requirements (e.g., functional, operational, dependability, and safety properties), which are expressed via a temporal logic formula. Also, they can be applied to partial models, therefore, it is not necessary to fully model the whole system behavior and its properties before analyzing its correctness. For this reason, they allow identifying incompleteness and inconsistencies in a set of requirements as well as flaws related to preliminary design concepts [8]. In such cases, counter-examples are provided, which can be used as clear guidance for the specification and design reiteration process. It is generally understood that model checking forces system engineers to translate system concepts (elaborated in natural language) into verifiable models, which can expose design issues before the satellite manufacturing phase, thus avoiding project cost increase and schedule drift [9]. Despite these promising advantages, model checking approaches to satellite and airspace projects (and more generally SoS) are still in their infancy and several further studies are needed. In Section V of this paper, some interesting related works from the literature are reported.

This paper proposes a model checking based approach highlighting the interfaces and the interdependencies among the different parts of complex systems (called processes) with the aim of exposing and verifying their correctness, completeness, and consistency as prescribed by the system requirements. Absence of deadlock or starvation of such concurrent processes can be also checked. As better detailed in Sections IV and V, this manuscript is one of the few examples where formal verification techniques are applied in the context of satellite development, and, contrary to what happens in the few available literature, it focuses on the concurrency and interactions among the different subsystems, while adopting complexity reduction and abstraction techniques to avoid the consequent state space explosion problem. This methodology exploits the flexibility provided

Manuscript received June 21, 2017; revised November 15, 2017; accepted January 7, 2018. *(Corresponding author: Antonella Santone.)*

V. Nardone, M. Tipaldi, D. Liuzza, and L. Glielmo are with the Department of Engineering, University of Sannio, Benevento 82100, Italy (e-mail: vnardone@unisannio.it; mtipaldi@unisannio.it; davide.liuzza@unisannio.it; glielmo@unisannio.it).

A. Santone is with the Department of Bioscience and Territory, University of Molise, Campobasso 86100, Italy (e-mail: antonella.santone@unimol.it).

Digital Object Identifier 10.1109/JSYST.2018.2793665

1937-9234 © 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

by the calculus of communicating systems (CCS) [10]. Such methodology has been applied to the modeling and the verification of a real satellite operational mode management (SOMM) specification, showing effective and promising results. All the satellite subsystems concurring in the SOMM along with their mutual interactions are modeled in CCS, and the related requirements expressed in mu-calculus. It is shown how it is possible to capture and analyze the parts of a system interested in the verification of a specific set of requirements. To do so, different levels of abstraction are needed and have to be balanced between description details and computational power demands. This way, the proposed approach can mitigate the state space explosion problem and scale up to complex SoS industrial projects. The use of CCS is a viable solution, thanks to the availability of efficient formal verification methodologies [11]. Furthermore, being a process algebra language, it has been proven valuable for the specification and the design of distributed systems, in particular for what concerns formal reasoning and rapid prototyping [7].

The paper is organized as follows. Section II outlines the main concepts of model checking. Section III presents the operational mode definition and management of real geostationary satellites, that is to say the SOMM. Section IV shows how to apply model checking techniques to the SOMM. Particular emphasis is given to the abstraction model checking. Finally, the paper wraps up with the related work in Section V and the conclusions in Section VI.

II. Preliminaries on Model Checking and CCS

This section introduces the basic concepts of model checking, a formal method widely used for verification of complex systems [7]. Model checking builds on rigorous mathematical foundations. Among other things, as stated in [12], *"formal methods are one of the highly recommended verification techniques for software development of safety-critical systems according to, e.g., the best practices standard of the International Electrotechnical Commission and standards of the European Space Agency."* Very promising verification techniques, accompanied by powerful software tools, have been developed by the formal methods community [7].

The inputs of a model checker are a model of the system to be analyzed and a set of properties, expressed as formulas of temporal logic, that are expected to hold on the system. The model checker either confirms that the properties hold or reports that they are violated. In the latter case, it provides a counter-example, i.e., a run that violates the property. This ability to generate counter-examples can be exploited to determine the cause of the violation. Thus, for applying model checking, two distinct formal notations for the definition of a system and of its properties are necessary. Such notations are examined in the following two sections.

A. Formal Notation for Defining Systems

Specification is the process of describing a system. It is assumed that the system behavior can be represented as a *Labeled Transition System (LTS)*. An LTS consists of a set of states and a set of transitions among those states. These transitions are labeled by actions and one state is designated as the initial state. More formally, an LTS is a quadruple $T = (S, \mathcal{A}, \longrightarrow, s)$, where

TABLE I
Standard Operational Semantics of CCS

Act	$\dfrac{}{\alpha.p \xrightarrow{\alpha} p}$	**Sum**	$\dfrac{p \xrightarrow{\alpha} p'}{p + q \xrightarrow{\alpha} p'}$ (and symmetric)
Rel	$\dfrac{p \xrightarrow{\alpha} p'}{p[f] \xrightarrow{f(\alpha)} p'[f]}$	**Par**	$\dfrac{p \xrightarrow{\alpha} p'}{p\|q \xrightarrow{\alpha} p'\|q}$ (and symmetric)
Con	$\dfrac{p \xrightarrow{\alpha} p'}{x \xrightarrow{\alpha} p'}\ x \stackrel{\text{def}}{=} p$	**Com**	$\dfrac{p \xrightarrow{l} p',\ q \xrightarrow{\bar{l}} q'}{p\|q \xrightarrow{\tau} p'\|q'}$
		Res	$\dfrac{p \xrightarrow{\alpha} p'}{p\backslash L \xrightarrow{\alpha} p'\backslash L}\ \alpha, \overline{\alpha} \notin L$

S is the set of states, $\mathcal{A}$ is the set of transition labels (actions), $s \in S$ is the initial state, and $\longrightarrow \subseteq S \times \mathcal{A} \times S$ is the transition relation.

For the purpose of mathematical reasoning, it is often convenient to represent the LTS algebraically in the form of processes [10]. To this aim, Milner's CCS can be used. CCS is one of the most well known process algebras, largely used for modeling complex systems. It contains basic operators to build finite processes, communication operators to express concurrency, and some notion of recursion to capture infinite behavior. The syntax of *processes* is the following:

$$p ::= \text{nil} \,|\, \alpha.p \,|\, p + p \,|\ p|p\ \,|\, p\backslash L \,|\, p[f] \,|\, x$$

where α ranges over a finite set of actions $\mathcal{A} = \{\tau, a, \bar{a}, b, \bar{b}, ...\}$. Input actions are labeled with "nonbarred" names, i.e., a, while output actions are "barred," i.e., $\bar{a}$. Two processes may interact whenever a process is prepared to perform some action a and the other one is prepared to trigger it via the barred action $\bar{a}$. Such actions are called *complementary actions*. The action $\tau \in \mathcal{A}$, called *internal action*, is usually used in the literature to indicate a generic internal action. It allows some level of abstraction in the description of the processes, since it can hide an arbitrarily complex sequence of actions whose details are kept private. The set L ranges over sets of *visible actions* $\mathcal{A} - \{\tau\}$ and can be used to create process communication via internal actions. Visible actions are used to define the system properties to be verified. The relabeling function f, in processes of the form $p[f]$, is a function, $f : \mathcal{A} \rightarrow \mathcal{A}$, such that the constraint $f(\tau) = \tau$ is respected. Moreover, x ranges over a set of *constant* names: each constant x is defined by a constant definition $x \stackrel{\text{def}}{=} p$.

The semantics of a CCS process is provided via a transition system: The transition is deduced applying a set of inference rules, driven by the syntax of the CCS processes. More formally, the standard *operational semantics* [10] is given by a relation $\longrightarrow \subseteq \mathcal{P} \times \mathcal{A} \times \mathcal{P}$, where $\mathcal{P}$ denotes the set of processes. $\longrightarrow$ is the least relation defined by the rules in Table I.

The semantics for CCS can be informally explained by induction over the structure of processes. nil represents a process that can do nothing. There is no rule for nil since it cannot evolve. The process $\alpha.p$ can perform the action α and thereby become the process p (rule Act). The process $p + q$ is a process that nondeterministically behaves either as p or as q (rule Sum). The operator $|$ expresses parallel composition. p and q may act independently: If the process p can perform α and become p', then $p|q$ can perform α and become $p'|q$, and similarly for q (rule

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

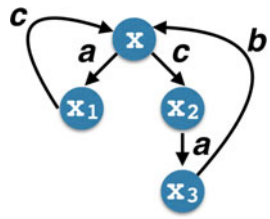


Fig. 1. $\mathcal{S}(x)$.

Par). Furthermore, p and q may also together engage in a communication whenever they are able to perform complementary actions. That is, if p can perform a visible action l and become p', and q can perform $\bar{l}$ and become q', then $p|q$ can perform τ and become $p'|q'$ (rule Com). If L is a set of visible actions, $p\backslash L$ is a process that behaves as p except that it cannot perform any of the actions (as well as the corresponding complementary actions) lying in L externally, although each pair of these complementary actions can be performed for communication internally (rule Res). The operator $[f]$ expresses the relabeling of actions. If p can perform α and become p', then $p[f]$ can perform $f(\alpha)$ and become $p'[f]$ (rule Rel). The behavior of the process x ($x \stackrel{\text{def}}{=} p$) is that of its definition p (rule Con).

Given a CCS process p, an LTS corresponding to the behavior of p can be built. This LTS is called *standard transition system* of p and is denoted by $\mathcal{S}(p)$. For example, consider the CCS process $x \stackrel{\text{def}}{=} c.a.b.x + a.c.x$. The standard transition system of x, i.e., $\mathcal{S}(x)$ is shown in Fig. 1.

B. A Formal Notation for Defining Properties

Temporal logic formulas provide a rigorous notation for defining properties. Indeed, they present constructs allowing to state in a formal way that, for instance, all scenarios will respect some properties at every step, or that some particular event will eventually happen, etc. The most noticeable examples are the *safety properties*, which state that an undesirable situation will never arise, and the *liveness properties*, which state that some actions will always be followed by some reactions.

This paper proposes an approach based on the *selective mu-calculus* logic [13]. It was defined with the goal of reducing the number of states of the transition systems in such a way that the reduction is driven by the formulas to be checked, in particular by their syntactic structure. The syntax of the selective mu-calculus is the following (K and R range over sets of actions, while Z ranges over a set of variables):

$$\varphi ::= \texttt{tt} \,|\, \texttt{ff} \,|\, Z \,|\, \varphi \vee \varphi \,|\, \varphi \wedge \varphi \,|\, [K]_R\, \varphi \,|\, \langle K \rangle_R\, \varphi \,| \\ \nu Z.\varphi \,|\, \mu Z.\varphi.$$

The satisfaction of a formula φ by a state s of a transition system is defined as follows.

Each state satisfies $\texttt{tt}$ and no state satisfies $\texttt{ff}$. A state satisfies $\varphi_1 \vee \varphi_2$ ($\varphi_1 \wedge \varphi_2$) if it satisfies φ_1 or (and) φ_2. $[K]_R\, \varphi$ and $\langle K \rangle_R\, \varphi$ are the selective modal operators. They require that the formula φ is satisfied immediately after the execution of an action of K, provided that it is not preceded by any action in $R \cup K$. More precisely: $[K]_R\, \varphi$ is satisfied by a state which, for every execution of a sequence of actions not belonging to $R \cup K$, followed by an action in K, evolves in a state obeying φ. $\langle K \rangle_R\, \varphi$ is satisfied by a state that can evolve to a state obeying φ by performing a specific sequence of actions not belonging to $R \cup K$ followed by an action in K. As in standard mu-calculus, a fixed point formula has the form $\mu Z.\varphi$ ($\nu Z.\varphi$). For an intuition of their meaning, consider the formulas φ_1: $\mu Z.\varphi \vee \langle a \rangle_\emptyset\, Z$ and φ_2: $\nu Z.\varphi \wedge [a]_\emptyset\, Z$. The formula φ_1 is satisfied by a state that can evolve to a state satisfying φ after a finite number of occurrences of action a (ignoring all other actions), while the formula φ_2 is satisfied by a state which satisfies φ along any path ending with a (ignoring all other actions).

TABLE II
SATISFACTION OF A CLOSED FORMULA BY A STATE

$s \not\models \texttt{ff}$		
$s \models \texttt{tt}$		
$s \models \varphi \wedge \psi$	iff	$s \models \varphi$ and $s \models \psi$
$s \models \varphi \vee \psi$	iff	$s \models \varphi$ or $s \models \psi$
$s \models [K]_R\, \varphi$	iff	$\forall s', \forall \alpha \in K, s \stackrel{\alpha}{\Longrightarrow}_{K \cup R} s'$ implies $s' \models \varphi$
$s \models \langle K \rangle_R\, \varphi$	iff	$\exists s', \exists \alpha \in K, s \stackrel{\alpha}{\Longrightarrow}_{K \cup R} s'$ and $s' \models \varphi$
$s \models \nu Z.\varphi$	iff	$s \models \nu Z^n.\varphi$ for all n
$s \models \mu Z.\varphi$	iff	$s \models \mu Z^n.\varphi$ for some n

where, for each n, $\nu Z^n.\varphi$ and $\mu Z^n.\varphi$ are defined as

$\nu Z^0.\varphi = \texttt{tt}$ $\qquad$ $\mu Z^0.\varphi = \texttt{ff}$

$\nu Z^{n+1}.\varphi = \varphi[\nu Z^n.\varphi/Z]$ $\qquad$ $\mu Z^{n+1}.\varphi = \varphi[\mu Z^n.\varphi/Z]$

where the notation $\varphi[\psi/Z]$ indicates the substitution of ψ for every free occurrence of the variable Z in φ.

The formal definition of the satisfaction of a closed formula φ by a state of a transition system $\mathcal{T} = (\mathcal{S}, \mathcal{A}, \longrightarrow, s)$ is given in Table II. It uses the transition relation $\Longrightarrow_I$, parametric with respect to $I \subseteq \mathcal{A}$, which ignores all noninteresting actions (i.e., those in $\mathcal{A} - I$).

Definition 2.1 ($\Longrightarrow_I$ relation): Let p be a CCS process and $I \subseteq \mathcal{A}$, the relation $\Longrightarrow_I \subseteq \mathcal{P} \times I \times \mathcal{P}$ is defined such that, for each $\alpha \in I$ and $p, q \in \mathcal{P}$:

$$p \stackrel{\alpha}{\underset{I}{\Longrightarrow}} q \quad \text{iff} \quad p \xrightarrow{\delta\alpha} q, \text{ where } \delta \in (\mathcal{A} - I)^*.$$

The notation $p \stackrel{\alpha}{\Longrightarrow}_I q$ expresses the fact that it is possible to pass from p to q by performing a (possibly empty) sequence of actions not belonging to I and then the action α in I. Note that $\Longrightarrow_{\mathcal{A}} = \longrightarrow$. A CCS process p satisfies φ, written $p \models \varphi$ if the initial state of $\mathcal{S}(p)$ satisfies φ.

Some examples of selective mu-calculus formulas are provided hereafter: $\psi_1 = [a]_{\{b\}}\texttt{ff}$: "it is not possible to perform an action a if an action b has not been previously performed;" $\psi_2 = \langle a \rangle_\emptyset \texttt{tt}$: "it is possible to perform an action a preceded by any action;" $\psi_3 = \nu Z.[a]_\emptyset(Z \wedge [a]_{\{b,c\}}\texttt{ff})$: "it always holds that, after an action a has occurred, a successive a cannot occur before either an action b or an action c has occurred."

Recall the transition system of Fig. 1. It holds that: $x \not\models \psi_1,\ x \models \psi_2,\ x \models \psi_3$.

III. Operational Mode Management in Geostationary Satellites

This section presents the operational mode definition and management for geostationary satellites during both the launch

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

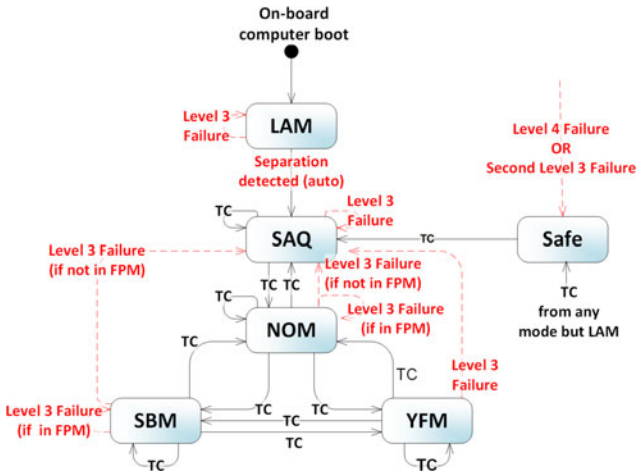


Fig. 2. Satellite modes and corresponding transitions (automatic transitions are in dashed red).

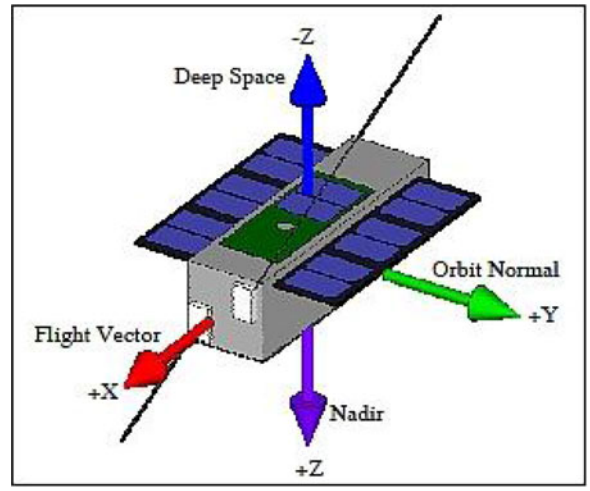


Fig. 3. Satellite axis orientation in nominal mode.

S/C Mode	LAM	SAQ				NOM		YFM	SBM		SAFE
AOCS Mode	STBY	STBY	SAM	GAM	OTM	FPM	SKM	FPM	SKM	FPM	SRM
Payload	SRV	non-op				Any	non-op	non-op	non-op	Any	SRV
Launch	X										
Transfer Orbit/Separation Sequence		X	X								X
Transfer Orbit/LEOP		X	X	X	X						X
GEO/On Station		X	X	X		X	X	X	X	X	X
GEO/Orbit Relocation		X	X	X	X			X	X	X	X
Disposal		X	X	X	X						X

Fig. 4. Satellite, AOCS, and payload mode combination in each mission phase (non-op=nonoperational).

and early orbit phase (LEOP) and the fully operational capability phase, that is to say, the geostationary phase (GEO). The presented approach draws on real satellite projects. Moreover, the requirements to be verified in Section IV are identified.

Starting from the launch phase up to its disposal, and in each mission phase, a satellite is featured by a well-defined set of modes [6]. A satellite mode can be regarded as a clearly identified configuration of both satellite platform and payload/instrument hardware and software items. Such cyber physical configuration of different subsystems has operational implications, meaning that each satellite mode allows a specific set of operations in order to fulfill its own operational objectives and requirements.

The conception of satellite mode management usually depends on the type of mission and its operational requirements. Geostationary satellites have continuous ground visibility with very small communication delay. Therefore, they can be operated in a quasi real-time fashion. However, they may have very high availability requirements since loss of services can have severe economical consequences [14]. As a rule, there can be two extreme cases, either the ground operators drive the complete satellite mode transition down to each HW or SW unit by executing a step-by-step procedure and uploading the related telecommands, or the ground operators send a telecommand to trigger a mode transition, with the satellite converting it into a series of on-board operations for the satellite subsystem reconfiguration. In the solution as illustrated hereafter, operational mode transitions are carried out by the ground segment through a well-defined procedure, which ends up with the telecommand to perform the actual satellite mode transition. Besides this, such mode transitions can be managed autonomously on board only in case of critical failures (level 3 or level 4 failures [14]) and a very limited number of nominal circumstances.

A. Satellite Mode Definition and Transition

The operational satellite modes are shown in Fig. 2. The satellite is in launch mode (LAM) from the moment of power provision from ground to the on-board batteries. At detection of the launcher separation signal by the on-board computer, the satellite enters automatically the sun acquisition mode (SAQ) and executes the separation sequence. Solar panels are oriented towards a specific reference position and the sun pointing has to be reached and maintained. In nominal mode (NOM), the satellite along its payload is fully operational. The satellite is three-axes stabilized ensuring the required high pointing accuracy of the +Z face towards the earth in order to provide mission product data and services (see Fig. 3). The yaw flip mode is required at each equinox to avoid direct sun illumination of the satellite −Y face. In stand-by mode (STBY), the satellite acts as an in-orbit spare not used for any operational service, but able to become operational within a short time frame. The satellite enters the safe mode under critical on-board anomaly detection, which cannot be autonomously recovered or which, though recovered, could present a risk later on if normal operations are continued. Return to normal operations is completely carried out under ground control.

As shown in Fig. 4, satellite operational modes are strictly interconnected to both the payload and the attitude and orbit control system (AOCS) modes throughout the complete mission. The AOCS operational modes are outlined in order to support the understanding of this paper. In STBY, all the sensors and the actuators are available, however, no closed loop actuation is performed by the satellite. The guided attitude mode (GAM) is designed to allow any kind of pointing and to perform small attitude manoeuvres, thus increasing the flexibility of operations of the satellite. The orbit transfer mode (OTM) gives the capability of performing the transfer to GEO

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

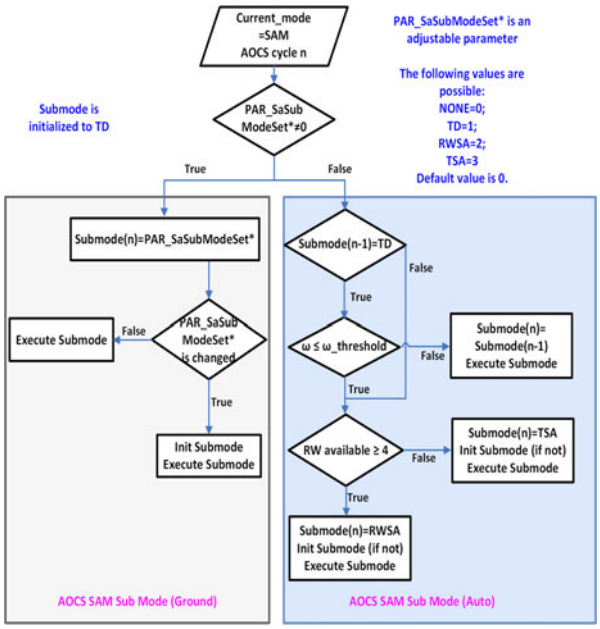


Fig. 5. AOCS submode transitions in SAM.

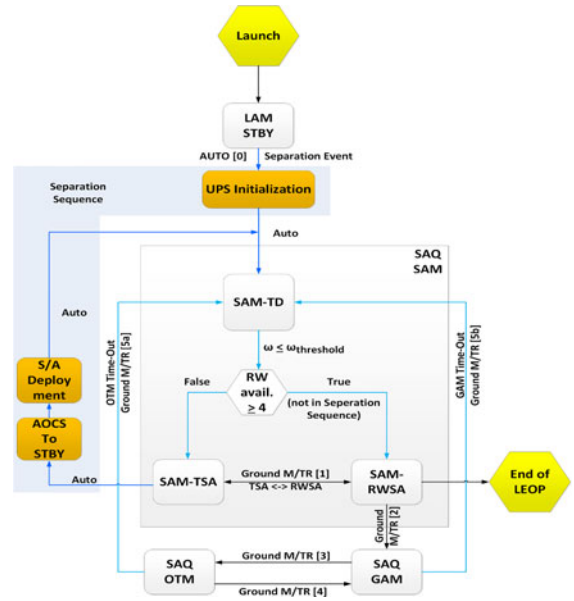


Fig. 6. Satellite mode management in LEOP.

using the liquid apogee engine, while maintaining the attitude compatible with this orbit correction. The station keeping mode (SKM) is entered to perform attitude control manoeuvres via the reaction wheels and orbit correction manoeuvres via the thrusters. The fine pointing mode (FPM) is used for precise satellite attitude manoeuvres. It is performed via the reaction wheels and is devoted to the payload operations. The survival mode (SRM) performs a stabilized and robust sun pointing in case of critical failure, independently of ground support.

The purpose of the sun acquisition mode (SAM) is to acquire a stable sun pointing attitude around the Z-axis with the −Z face pointing towards the sun. The complete operation is accomplished via a hybrid thrusters and reaction wheels strategy, very suitable for large angle and rapid satellite reorientation [15]. Thrusters can provide a larger control torque (to reduce the initial high angular momentum), whereas reaction wheels can be used to achieve high control accuracy. This is why SAM consists of three submodes: Thruster detumbling (TD), thruster sun acquisition (TSA), and reaction wheel sun acquisition (RWSA). The logic of submode selection in SAM is shown in Fig. 5. At launch, the parameter PAR_SaSubModeSet is set to NONE, and the AOCS SAM algorithm autonomously selects the submode TD. As soon as the rates are below the threshold, SAM switches from TD to either TSA or RWSA in the next AOCS cycle. Since the satellite is launched with all the reaction wheels in off condition, the TSA submode is automatically entered. On the other hand, during the GEO phase, the parameter PAR_SaSubModeSet is overwritten, thus the AOCS enters the commanded submode and stays there (see the left side of the Fig. 5).

B. Mode Transitions in LEOP

The satellite and AOCS mode management during the LEOP phase is shown in Fig. 6. In particular, the following operations are carried out: The separation sequence, the preparation for the satellite transfer into the GEO orbit, and the actual transfer. The separation sequence aims at bringing the satellite autonomously into a stable configuration with the solar arrays deployed, the −Z face pointing to the sun, and the propulsion subsystem initialized. Afterward, the AOCS can enter GAM, which is used as an intermediate mode to achieve the necessary attitude conditions at the beginning of orbit transfer manoeuvres. The latter are performed with the AOCS mode set to OTM. With the exception of the separation sequence, the complete LEOP is handled via TC by the ground operators. The following requirements are identified.

LEOP-1: From the separation event on and for the entire LEOP phase, the satellite shall stay in SAQ mode.

LEOP-2: During the propulsion initialization procedure, the AOCS subsystem shall enter automatically (and stay in) the STBY mode.

LEOP-3: The AOCS subsystem shall start the satellite rate damping procedure with the SAM submode initialized to TD, and then shall automatically switch to TSA once the satellite rate is lower that a specific threshold.

LEOP-4: During the solar array deployment, the AOCS subsystem shall enter automatically (and stay in) the STBY mode. After the completion of the solar array deployment, the AOCS subsystem shall enter automatically the SAQ-TSA mode (this event ends the separation sequence).

LEOP-5: The AOCS SAM shall enter automatically the RWSA submode once four reaction wheels are switched on via TC.

LEOP-6: Both the transitions to GAM and to OTM shall be performed via TC. The LEOP phase shall end after the completion of the OTM manoeuvres with the satellite/AOCS modes set to SAQ/SAM.

C. Mode Transitions in GEO

The satellite and AOCS mode transitions during the GEO phase are shown in Fig. 7. In GEO, the main objective is to

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

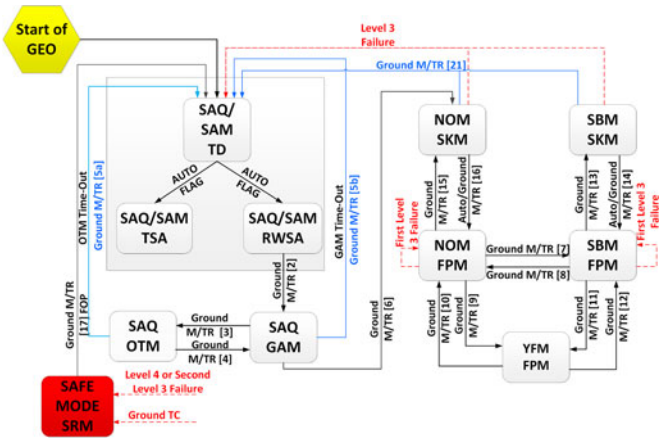


Fig. 7. Satellite mode management in GEO (transitions due to failure conditions are in dashed red).

fully operate the satellite and its payload in order to provide high-quality mission product data and services. The first transition from SAQ into NOM mode is executed during the satellite commissioning phase after its arrival in the GEO location, that is to say, after the end of the LEOP phase. Apart from a few exceptions, all the operations are performed by the ground operators during the GEO phase. The following requirements are identified.

GEO-1: Ground operators shall execute the following mode transitions to bring the satellite into a fully operational status: SAQ/SAM → SAQ/GAM → NOM/SKM → NOM/FPM.

GEO-2: Whenever level 3 or level 4 failures occur, the satellite shall enter automatically either the SAQ or the Safe mode as depicted in Fig. 7.

GEO-3: Whenever the AOCS is in FPM and at the first occurrence of a level 3 failure, the satellite shall stay in NOM/FPM.

GEO-4: The recovery procedure from safe mode up to NOM mode shall be performed by the Ground operators according to the following pattern: SAFE → SAQ/SAM → SAQ/GAM → NOM/SKM → NOM/FPM.

GEO-5: In SKM mode, once both the satellite angular momentum and the angular rate are lower than a specific threshold, an automatic transition into FPM mode shall be performed. If the dynamic conditions for FPM are not reached when the SKM time-out expires, the satellite shall remain in SKM, with constantly rechecking such conditions. If they are still not met after a longer time-out, an autonomous transition into SAQ/SAM shall be carried out.

GEO-6: The satellite and AOCS modes shall be congruent as shown in Fig. 7.

IV. Formal Verification of the Satellite Operational Mode Management

This section presents the abstraction reduction approach for the SOMM system formal verification. In the first place, as explained in Section II, this means building a formal specification of the SOMM system behavior, encoding the properties to be verified via temporal logic definitions, and having a formal verification environment. The following sections address such topics in details, provide the rationale of the proposed abstraction technique by comparing it with other approaches, and finally present the obtained results. In order to be concise and highlight to the reader the adopted methodology, in what follows only a few of all the verified properties are reported as illustrative examples.

TABLE III
CCS *SAT_LAM_MODE SAT_SAQ_MODE* AND *SEPARATION_SEQ* SPECIFICATIONS

```
SAT_LAM_MODE    = satelliteToLam.SAT-
                  LAM.'lamOk.SAT_SAQ_MODE
SAT_SAQ_MODE    = satelliteToSaq.SAT-
                  SAQ.'saqOk.SAT_SAQ_MODE
SEPARATION_SEQ  = startSepSeq.'SAT-SAQ.saqOk.
                  'aocsToSbty.aocsToStbyOk.
                  startPropInit.
                  PropInitOk.'aocsToSam.aocsToSamOk.
                  AOCS-SAM-TSA-OK.'AOCS-STBY.
                  AOCS-STBY-OK.SOLARARRAYDEPLOY.
                  SOLARARRAYDEPLOY-
                  OK.ENDSOLARARRAYDEPLOY.'aocsToSam.
                  aocsToSamOk.AOCS-SAM-TSA-
                  OK.'endSepSeq.nil
```

A. Formal Model Definition

The first step consists in specifying each component of the SOMM system by using the CCS process algebra based language. Each component of the SOMM involves complex interactions. Fig. 8 shows a view of all the processes (represented as boxes) and related interactions involved in the LEOP phase. Interactions are denoted by means of lines joining the involved processes with actions and their direction.

All the components have been specified in CCS. Table III shows only the CCS *SAT_LAM_MODE*, *SAT_SAQ_MODE* and *SEPARATION_SEQ* specifications in order to give the reader the flavor of the approach followed. (Note: for tool notation, output actions are denoted as $'a$ instead of $\overline{a}$ and $x = p$ means $x \stackrel{\text{def}}{=} p$). From a textual specification it is possible to automatically generate the corresponding labeled transition system, which can be successively used for model checking. Textual syntax is preferred since it is more suitable to proof-writing and formal reasoning, as well as to the description of large-scale systems. Graphical notations are anyway complementary and can be used by user-friendly tools.

B. Properties Definition

The second issue concerns the expression of the required properties listed in Sections III-B and III-C by means of the selective mu-calculus. Table IV shows the translation of a few properties into selective mu-calculus logic to give the reader the idea of the approach followed. In particular, as for the SOMM in LEOP, the LEOP-1 (in the following referred to as φ_1) and the LEOP-4 (in the following referred to as φ_2) are selected, whereas, as for the SOMM in GEO, the GEO-4 (in the following referred to as φ_3) is chosen.

From a more formal standpoint, the meaning of φ_1 is: "every time *SAT-SAQ* is performed, no *SAT-LAM* can be executed, stating that the system stays in *SAQ* mode."

The meaning of φ_2 is: "*SOLARARRAYDEPLOY* cannot be performed if *AOCS-STBY* has not been previously executed, and whenever *AOCS-STBY* occurs then no action belonging to

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

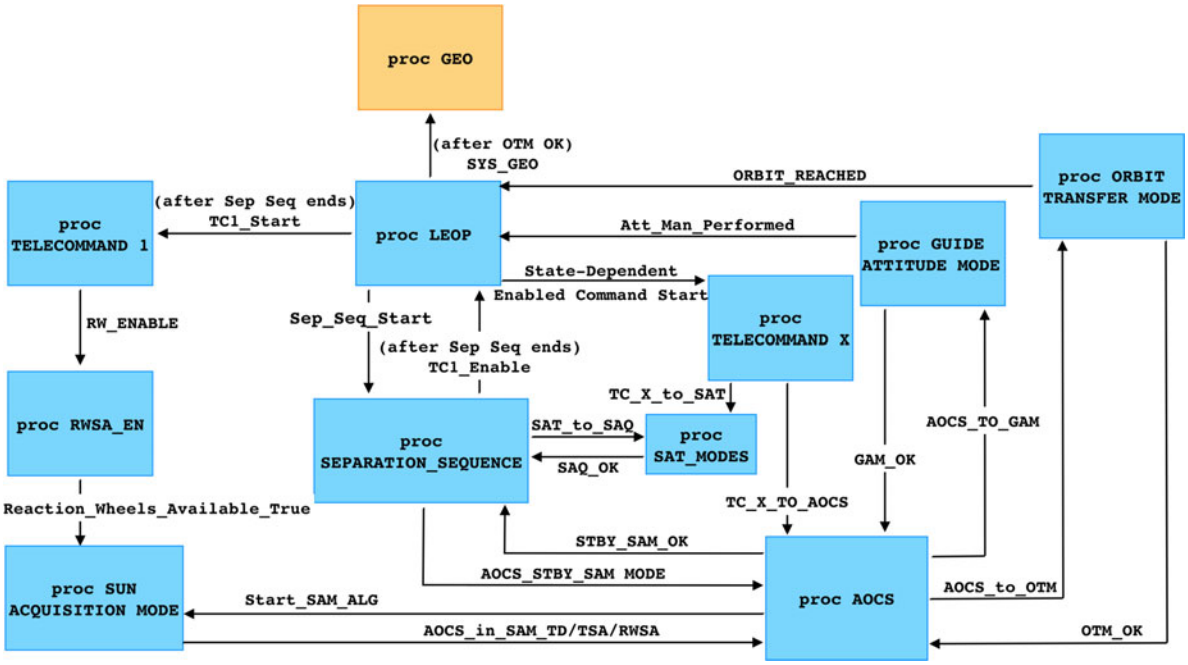


Fig. 8. Graphic representation of the interactions among all the LEOP processes involved in SOMM.

TABLE IV
EXAMPLE OF SELECTIVE MU-CALCULUS FORMULAS

Requirement	Formula
LEOP-1	$\varphi_1 = [SAT\text{-}SAQ]_\emptyset\,[SAT\text{-}LAM]_\emptyset\,\mathtt{ff}$
LEOP-4	$\varphi_2 = \psi_1 \wedge \psi_2 \wedge \psi_3$ $\psi_1 = [SOLARARRAYDEPLOY]_{\{AOCS\text{-}STBY\}}\,\mathtt{ff}$ $\psi_2 = [AOCS\text{-}STBY]_\emptyset[AOCS\text{-}SAM\text{-}TD,\ AOCS\text{-}SAM\text{-}TSA,\ AOCS\text{-}SAM\text{-}RWSA,\ AOCS\text{-}GAM,$ $AOCS\text{-}OTM]_{\{ENDSOLARARRAYDEPLOY\}}\mathtt{ff}$ $\psi_3 = [ENDSOLARARRAYDEPLOY]_\emptyset\,\langle AOCS\text{-}SAM\text{-}TSA\rangle_\emptyset\,\mathtt{tt}$
GEO-4	$\varphi_3 = \langle SAFE\text{-}SRM\text{-}MODE\rangle_\emptyset\,\langle TC17\rangle_\emptyset\,\langle TC2\rangle_\emptyset\,\langle TC6\rangle_\emptyset\,\langle TC14\rangle_\emptyset\,\mathtt{tt}$

the set

$$\{AOCS\text{-}SAM\text{-}TD,\ AOCS\text{-}SAM\text{-}TSA,\ AOCS\text{-}SAM\text{-}RWSA,\ AOCS\text{-}GAM,\ AOCS\text{-}OTM\}$$

can be performed before *ENDSOLARARRAYDEPLOY*; after the occurrence of *ENDSOLARARRAYDEPLOY* (i.e., at the end of the solar array deployment), a transition into *AOCS-SAM-TSA* shall be performed."

The meaning of φ_3 is: "when the system is *SAFE-SRM-MODE*, it is possible to execute in sequence *TC17* (i.e., the system enters the *SAQ/SAM* mode), *TC2* (i.e., the system enters the *SAQ/GAM*), *TC6* (i.e., the system enters the *NOM/SKM* mode), and finally *TC14* (i.e., the system enters the *NOM/FPM*)."

C. Formal Verification via Selective Mu-Calculus Based Abstraction Technique

Formal methods are difficult to be easily scaled up to complex projects due to the state space explosion problem, i.e., the state space grows exponentially with the number of concurrent processes. Indeed, the parallelism among the processes of a system leads to a number of reachable states which may become very large, in some cases on the order of millions or billions of states. When the number of states is too large to fit in the main memory of a computer, verification quickly breaks down. This is what happened with the formal verification environment CAAL used in this paper to verify the above specifications. CAAL (Concurrency Workbench, Aalborg Edition) [16] is one of the most popular formal verification environments for verifying concurrent systems, and supports several different specification languages, including the CCS. In CAAL, the verification of temporal logic formulas is based on model checking [7]. Unfortunately, the SOMM has a number of states so huge that cannot be handled by CAAL as well as by other model checkers. Several approaches in the literature have been developed to solve or mitigate the state space explosion problem. This paper exploits a methodology proposed by one of its authors [13]. It is an abstraction reduction technique based on selective mu-calculus logic. The basic characteristic of the selective mu-calculus is that the actions relevant for checking a formula φ are the ones explicitly mentioned in the modal operators used in the formula itself.

The set $\mathcal{O}(\varphi)$, namely ρ, can be defined as the set of the *occurring actions* of a formula φ. In [13], ρ-equivalence is defined,

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

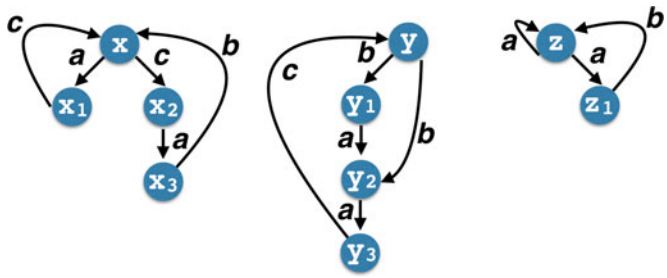


Fig. 9. Labeled transition systems for x, y, and z.

formally characterizing the notion of "the same behavior with respect to a set ρ of actions." It is proved that *"two transition systems are ρ-equivalent if and only if they satisfy the same set of formulas with occurring actions in ρ."* As a consequence, a formula of the selective mu-calculus with occurring actions in ρ can be checked on any transition system ρ-equivalent to the initial one. Thus, improvements in model checking can be obtained by minimizing the transition system with respect to the actions in ρ. Obviously, the degree of reduction depends on the size of ρ with respect to the size of the whole set $\mathcal{A}$ of actions.

Example 4.1: Consider the transition systems in Fig. 9 and the formulas: $\psi_1 = [a]_{\{b\}}\,\mathtt{ff}$ and $\psi_2 = \langle a \rangle_{\emptyset}\,\mathtt{tt}$.

The process x is $\{a, b\}$-equivalent to z, therefore the two processes give the same truth value for the formulas containing only actions in $\{a, b\}$. For example, they both satisfy ψ_2, while they do not satisfy ψ_1. Note that $\mathcal{O}(\psi_1) = \{a, b\}$ and $\mathcal{O}(\psi_2) = \{a\} \subseteq \{a, b\}$. Clearly, it is more efficient to verify both ψ_1 and ψ_2 on the transition system corresponding to the process z, which has only two states and three transitions, rather than using the transition system of the process x, which has four states and five transitions. On the contrary, the process y is not $\{a, b\}$-equivalent to z. In fact, differently form the process z, the process y can perform an action b without performing an action a. In fact, it holds that y satisfies ψ_1, while z does not.

In [17], a tool has been developed implementing the selective mu-calculus based abstraction reduction presented in [13]. Given a CCS process p and a set of interesting actions ρ occurring in a formula φ to be checked, it transforms p into another process q, corresponding to a smaller transition system than the original one, on which φ can be equivalently checked. The tool is based on a set of syntactic transformations rules. The output of the tool is in the format of the CAAL tool: a file .ccs, containing the reduced CCS process. Thus, both the construction of the transition system and the checking of the properties can be performed by using the CAAL formal verification environment.

D. Rationale of the Proposed Abstraction Reduction Technique

In literature, several approaches to reduce the state explosion problem have been proposed. In general, all the approaches have their limitations and their benefits depending on the application field. Thus, it is necessary to select the best methodology according to the system to be verified. The advantages of the proposed selective mu-calculus based abstraction for airspace applications are highlighted with respect to other three well known reductions techniques: Partial order, symbolic, and compositional.

The partial order approach to model checking [18] considers only a representative among all the interleavings of actions generated by a parallel composition. Unfortunately, the properties well handled by this approach do not concern precedence relations between actions. Indeed, in the proposed approach, the properties described in Sections III-B and III-C concern precedence relations between actions and are described by selective mu-calculus formulae which induce a consistent reduction of the state space of the transition system.

In the symbolic model checking [19], the state space is represented symbolically by a logical formula captured using a binary decision diagram (BDD) [20]. This technique has been proved especially successful in verifying hardware designs with regular logical structures. For such systems, BDD representations can reduce the state space from exponential order of the number of state variables to linear. This paper proposes an alternative to symbolic model checking, since system-level specifications lack of this regularity [21], and thus no significant reduction of the state space can be achieved.

Compositional techniques verify the components of the system individually, and then piece together the results to conclude that the original system is correct [22]–[24]. Compositional techniques suffer of several problems. Some techniques require user intervention and manual efforts, others consider only a subset of a temporal logic, and sometimes properties can be complex due to the environment specification. Differently, the proposed approach is completely automated, requires no user intervention, handles full selective mu-calculus logic, and obtains relevant reduction for properties concerning precedence relations between actions.

E. Formal Verification: Experimental Results

As said before, it is worth highlighting that the CAAL, due to the relevant number of states, is not able to give an answer on the correctness of the SOMM requirements as they are. Therefore, no verification can be performed at this stage, thus confirming the importance of the reduction approach. So, the abstraction reduction methodology described in Section IV-C is applied to verify the SOMM requirements. The tool detailed in [17] can transform the SOMM into a specification equivalent smaller process, where the reduction is driven by the actions occurring in the formulas. Finally, the properties on the reduced specification can be checked.

It holds that the sets of the occurring actions of each of the properties described above are

$$\rho_1 = \mathcal{O}(\varphi_1) = \{SAT\text{-}SAQ, SAT\text{-}LAM\}$$

$$\begin{aligned}\rho_2 = \mathcal{O}(\varphi_2) = \{&SOLARARRAYDEPLOY, AOCS\text{-}STBY,\\ &AOCS\text{-}SAM\text{-}TD, AOCS\text{-}SAM\text{-}TSA,\\ &AOCS\text{-}SAM\text{-}RWSA, AOCS\text{-}GAM,\\ &AOCS\text{-}OTM, ENDSOLARARRAYDEPLOY\}\end{aligned}$$

$$\begin{aligned}\rho_2 = \mathcal{O}(\varphi_3) = \{&SAFE\text{-}SRM\text{-}MODE, TC17, TC2,\\ &TC6, TC14\}.\end{aligned}$$

For example, by applying the proposed methodology with ρ_1 on *SAT_LAM_MODE*, *SAT_SAQ_MODE*, and *SEPARATION_SEQ* CCS processes of Table III, the reduced processes

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE V
RESULTS FOR THE SOMM

	Total			φ_1-reduction			φ_2-reduction			φ_3-reduction		
	states	trans.	time	states	trans.	time	states	trans.	time	states	trans.	time
SOMM	-	-	-	275	386	0.109	310	445	0.133	1296	3385	1.077

TABLE VI
REDUCED CCS *SAT_LAM_MODE*, *SAT_SAQ_MODE* AND *SEPARATION_SEQ* SPECIFICATIONS

```
SAT_LAM_MODE    = satelliteToLam.SAT-LAM.
                  SAT_SAQ_MODE
SAT_SAQ_MODE    = satelliteToSaq.SAT-SAQ.
                  SAT_SAQ_MODE
SEPARATION_SEQ  = startSepSeq.'SAT-SAQ.satOk.
                  'aocsToSbty.aocsToStbyOk.
                  'aocsToSam.aocsToSamOk.
                  AOCS-SAM-TSA-OK.'AOCS-STBY.
                  AOCS-STBY-OK.'aocsToSam.
                  aocsToSamOk.AOCS-SAM-TSA-OK.
                  'endSepSeq.nil
```

shown Table VI are obtained. It is important to note that, thanks to this methodology, several actions from the initial CCS specifications are eliminated.

Experiments have been executed on a 64 bit, 2.67 GHz Intel i5 CPU equipped with 8 GB of RAM and running Gentoo Linux. Table V shows the number of states and transitions for the φ_i-reductions, $i \in \{1, 2, 3\}$, obtained by applying the property-based methodology to the SOMM CCS processes with ρ_i, for each $i \in \{1, 2, 3\}$. It is worth noting that it is possible to verify the system since the reduction on both the states and the transitions is significant. Table V also shows the time (measured in seconds) employed to check the three selected formulas with the proposed formula-based methodology. All the properties of Section III have been proved and they have returned the true value. The proposed methodology has been applied to the set of all the given specifications via the preliminary conversion of the SOMM into a CCS and the formulas-driven abstractions described before.

V. RELATED WORK

Verifying SoS represents one of the current biggest challenges in system engineering. Indeed, the complexity of modern engineering systems is increasing at levels not easy of even not possible to be fully understood by human operators. The interactions among several physical and cyber components may arise unexpected complex behaviors, leading to failures difficult to predict, pushing the system toward unsafe working boundaries or leaving hidden vulnerabilities. Also, the design itself of the SoS is often difficult, requiring late verification, expensive testing, and possibly redesigning phases before having a reasonable certainty that the overall system performs according to the requirements. In view of this, exploring methodological and rigorous tools to address SoS formal requirement analysis, behavior, and integration is a recent trend in the literature, with some papers offering interesting approaches and considerations. It is however important to notice that formal requirement engineering and verification for complex SoS are in their infancy and both models and techniques need to be developed [25]. Nowadays, the lack of these rigorous methods still forces the majority of the industrial development and verification phases, including high-tech airspace, automotive, and defense sectors, to pursue minor incremental changes with respect to current practices and consolidated processes. Such an approach needs substantial improvements in the future, to allow a widespread diffusion of complex cyber-physical SoS engineering products. In what follows, we briefly revise some works where formal methodologies are applied to SoS design or verification. We also provide a dedicated section on airspace applications on such methodologies.

A. Formal Methods for SoS Requirements and Verification

With respect to general SoS architectures, in [26], an integrated design flow and verification tool chain is developed within the META research program. The latter was devoted to expedite the product development and deployment timeline of US defense systems. In such a work, the authors investigate the overall design space and formally address the functional and safety requirements in the early design stages of SoS. A seven steps methodology is presented exploiting a generative grammar approach on a components model library to ensure that the design space of the SoS is adequately explored and a further formal verification based on PRISMATIC model checker (a probabilistic model checker based on PRISM) on the selected candidate designs is performed. After that, performance aspects are validated via Monte Carlo simulations or perturbation approaches. In [25], a contract-based design methodology is presented, where design requirements are verified and refined progressively. Contracts are formalizations of assumptions (that the environment has to satisfy when acting on each component) and guarantees (that each component has to ensure when assumptions are satisfied). Specifically, elements integration is formalized via horizontal contracts, while hierarchical abstraction is formalized via vertical contracts. Requirements modeling and verification can be then performed at different abstraction layers. A fully formal contract framework based on temporal logics is also presented in [27]. This framework considers synchronous or asynchronous decompositions of a SoS into subcomponents to generate a set of proof obligations. Both the contracts and the proof obligations are expressed in temporal logic formulas. The latter are then verified via a formal model checker, guaranteeing the verification of the overall SoS. Different formal languages are supported by the method. Formal methods are also adopted in [28]. Specifically, cyber-physical systems are modeled with the Chi formalism, a hybrid process algebra for modeling discrete/continuous hybrid

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

systems. They are then formally verified with the bounded model checker PHAVer [29]. From formal methods techniques, runtime verification is derived in [30]. The author proposes the idea of runtime verification for those systems that cannot be totally verified solely in the early stage design. An example is given by the adaptive systems, that push the synthesis phase at runtime. In [31], several potential strategies for integration, verification, and validation of SoS are discussed. Among them, formal methodologies are briefly revised, although the paper focuses, as well as [32], more on graph theoretical approaches. An integrative approach for verification of SoS is addressed in [33] and based on the formal logic of objects and processes of Leśniewski [34]. The framework allows to address requirements verification for both systems and SoS. A probabilistic model checking of SoS is studied in [35], where the authors propose a modeling scheme able to represent different types of SoS with respect to the degree of authority the overall system has on its component subsystems. The model checking is then conducted with PRISM. The same tool is also used in [36] and [37] where the problems of dynamic management and resource allocation and the verification of the Chinese train control system are studied, respectively. In [38], the authors consider formal verification for an audio and video SoS.

B. Formal Methods for Spacecraft Design

Due to the fact that formal methods for SoS design and verification is still in an early stage of research, and several approaches and specific techniques may possibly be adopted, their exploitation for airspace applications has been addressed with few dedicated studies. Indeed, as for the utilization of formal method techniques for spacecraft design, the project correctness, modeling and performance of aerospace systems (COMPASS) is drawing a lot of attention within the European space community [5]. It provides a rich set of tools and allows space system engineers to express the spacecraft units and their dependability aspects via the tailoring of the architecture analysis & design language. The COMPASS tool set utilizes symbolic and probabilistic model checking techniques for the analysis of requirements related to functional correctness, safety, dependability, and performance.

Special attention is also being given to formal verification methods in order to address complex system and mission requirements, such as advanced spacecraft autonomy concepts implemented via the so-called model-based autonomy [14]. In such a case, their verification can be accomplished via a compositional-based formal approach, which is suitable for the needs of model-based systems [39]. In [40], a seamless integration of a spacecraft simulation framework and model checking techniques is proposed to verify mission lifetime requirements. [41] shows that it is possible to use the early design data of a spacecraft and to transform it into a state model. This state model can be used to understand and in particular verify that the designed spacecraft fits to the mission requirements. In [42], formal methodologies are adopted together with a contract based approach. The design state space is explored via subsequent refinement steps of abstraction layers and the requirements are expressed with the formal languages of linear temporal logic and signal temporal logic. The different requirements over the several subsystems of the SoS may be separately verified and combined in a compositional way via contracts. Such methodology is adopted to synthesize an aircraft power system. In [43], the global earth observation system of systems is formally verified via a previous conversion of the SoS model into a colored Petri net. Architectural requirements of complex SoS are addressed in the early stages of the development process in [44], where object-oriented models are augmented with time-series temporal logic requirements. An application to a ballistic missile defense system is also considered. Thanks to their strong expressive power, verification of properties expressed in temporal logics has also attracted the increasing interest of the control system engineering community. With respect to airspace control applications, few interesting recent works have been produced. Specifically, in [45], the control software for the power management subsystem is synthesized, while in [46], a controller is designed to satisfy rich temporal specification on the fuel imbalance between the two tanks in an aircraft. Moreover, in [47] formal methods are adopted to properly design and verify the safety-critical air management system of an aircraft in the presence of not fully observable states.

The ECSS-E-ST-40CA [48] standard suggests the adoption of formal methods to prove the completeness and the consistency of the spacecraft on-board software requirements independently from their implementation. An interesting usage of formal methods can be found in [49], where the symbolic model checking can be adopted to verify the correctness of spacecraft on-board software implementation against a given specification.

VI. Conclusion

This paper shows how the verification of space systems requirements can be performed via model checking, a formal verification method based on abstract behavioral models. More specifically, the proposed approach highlights the interdependencies among the different parts of space systems with the aim of exposing and verifying their correctness, completeness, and consistency as prescribed by the addressed system requirements. Such methodology is based on the usage of CCS and has been applied to the verification of a real SOMM specification.

Thanks to its intrinsic nature in abstracting implementation details, model checking can be used in the early phases of space projects, when detailed information is missing. Being based on strict mathematical reasoning, complex requirements can be proved on a conceptual level before starting the satellite manufacturing phase. Moreover, it has been also explained how to capture and analyze the parts of a system involved in the verification of a specific set of its requirements, thus solving the state space explosion problem, which is a primary issue associated with the model checking technique.

Encouraged by the results of the proposed methodology, the authors of this paper plan to apply such technique to a more complex set of requirements, for instance the verification of the spacecraft failure management concept. In such a case, the abstraction approach will be of fundamental importance to capture all the components concurring to manage a specific class of on-board failures. Despite the encouraging results and benefits, formal methods still remain to be institutionalized in the space systems development process. In this regard, a crucial element is the adoption of an industrialized model-checking framework, which should include the following features: Modeling

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

languages (e.g., SySML) to express the SoS as the composition of finite state transition machines, tools for translating them into the input language of the model checking engine, tools for visualizing the results and easily browsing the counter-examples, and abstraction options for reducing the large models to a size suitable for feasible verification.

ACKNOWLEDGMENT

The authors gratefully acknowledge the valuable contribution of M. Suppa for his help in implementing the CCS models for the SOMM specification during his Master Thesis.

REFERENCES

[1] "ECSS-E-ST-10C: Space Engineering - Systems Engineering General Requirements," European Cooperation for Space Standardization, Zurich, The Netherlands, 2009.

[2] M. A. Viscio, N. Viola, R. Fusaro, and V. Basso, "Methodology for requirements definition of complex space missions and systems," *Acta Astronaut.*, vol. 114, pp. 79–92, 2015.

[3] S. Chaieb, M. Wegerson, B. Kading, J. Straub, R. Marsh, and D. Whalen, "The openorbiter cubesat as a system-of-systems (SoS) and how sos engineering (SoSE) aids cubesat design," in *Proc. 10th Syst. Syst. Eng. Conf.*, 2015, pp. 47–52.

[4] A. Golkar and I. L. i Cruz, "The federated satellite systems paradigm: Concept and business case evaluation," *Acta Astronaut.*, vol. 111, pp. 230–248, 2015.

[5] M. Bozzano *et al.*, "Spacecraft early design validation using formal methods," *Reliab. Eng. Syst. Saf.*, vol. 132, pp. 20–35, 2014.

[6] J. Eickhoff, *Onboard Computers, Onboard Software and Satellite Operations: An Introduction*. New York, NY, USA: Springer, 2012.

[7] E. M. Clarke, O. Grumberg, and D. Peled, *Model Checking*. Cambridge, MA, USA: MIT Press, 2001.

[8] D. Aceituna, D. Hyunsook, and S. Srinivasan, "A systematic approach to transforming system requirements into model checking specifications," in *Proc. 36th Int. Conf. Softw. Eng.*, 2014, pp. 165–174.

[9] M. Esteve, J. P. Katoen, V. Nguyen, B. Postma, and Y. Yushtein, "Formal correctness, safety, dependability, and performance analysis of a satellite," in *Proc. Int. Conf. Softw. Eng.*, 2012, pp. 1022–1031.

[10] R. Milner, *Communication and Concurrency* (PHI Series in computer science). Englewood Cliffs, NJ, USA: Prentice-Hall, 1989.

[11] G. Anastasi, A. Bartoli, N. D. Francesco, and A. Santone, "Efficient verification of a multicast protocol for mobile computing," *Comput. J.*, vol. 44, no. 1, pp. 21–30, 2001.

[12] C. Baier and J.-P. Katoen, *Principles of Model Checking* (Representation and Mind Series) Cambridge, MA, USA: MIT Press, 2008.

[13] R. Barbuti, N. D. Francesco, A. Santone, and G. Vaglini, "Selective mu-calculus and formula-based equivalence of transition systems," *J. Comput. Syst. Sci.*, vol. 59, no. 3, pp. 537–556, 1999.

[14] M. Tipaldi and B. Bruenjes, "Survey on fault detection, isolation, and recovery strategies in the space domain," *J. Aerosp. Inf. Syst.*, vol. 12, pp. 235–256, 2015.

[15] D. Ye, Z. Sun, and S. Wu, "Hybrid thrusters and reaction wheels strategy for large angle rapid reorientation with high precision," *Acta Astronaut.*, vol. 77, pp. 149–155, 2012.

[16] J. R. Andersen *et al.*, "CAAL: concurrency workbench, aalborg edition," in *Theoretical Aspects of Computing—ICTAC 2015*. (vol. 9399, Lecture Notes in Computer Science). New York, NY, USA: Springer, 2015, pp. 573–582.

[17] R. Barbuti, N. D. Francesco, A. Santone, and G. Vaglini, "Reduced models for efficient CCS verification," *Formal Methods Syst. Des.*, vol. 26, no. 3, pp. 319–350, 2005.

[18] D. A. Peled, "Introduction: Special issue on partial order in formal methods," *Formal Methods Syst. Des.*, vol. 26, no. 1, pp. 5–6, 2005.

[19] E. M. Clarke, K. L. McMillan, S. V. A. Campos, and V. Hartonas-Garmhausen, "Symbolic model checking," in *Proc. 8th Int. Conf. Comput. Aided Verif.*, 1996, pp. 419–427.

[20] R. E. Bryant, "Graph-based algorithms for boolean function manipulation," *IEEE Trans. Comput.*, vol. 35, no. 8, pp. 677–691, Aug. 1986.

[21] S. Cheung and J. Kramer, "Checking safety properties using compositional reachability analysis," *ACM Trans. Softw. Eng. Methodol.*, vol. 8, no. 1, pp. 49–78, 1999.

[22] C. S. Pasareanu, M. B. Dwyer, and M. Huth, "Assume-guarantee model checking of software: A comparative case study," in *Proc. Int. SPIN Workshop Model Checking Softw.*, 1999, vol. 99, pp. 168–183.

[23] R. Kaivola, "Compositional model checking for linear-time temporal logic," in *Proc. Comput, Aided Verification, 4th Int. Workshop.*, Montreal, QC, Canada, 1992, pp. 248–259.

[24] H. R. Andersen, C. Stirling, and G. Winskel, "A compositional proof system for the modal mu-calculus," in *Proc. 9th Annu. Symp. Logic Comput. Sci.*, Paris, France, 1994, pp. 144–153.

[25] A. Sangiovanni-Vincentelli, W. Damm, and R. Passerone, "Taming Dr. Frankenstein: Contract-based design for cyber-physical systems," *Eur. J. Control*, vol. 18, no. 3, pp. 217–238, 2012.

[26] S. Uckun, "META II: Formal co-verification of correctness of large-scale cyber-physical systems during design," Palo Alto Research Center, Palo Alto, CA, USA, Tech. Rep. AFRL-RZ-WP-TR-2011-2110, vol. I, 2011.

[27] A. Cimatti and S. Tonetta, "Contracts-refinement proof system for component-based embedded systems," *Sci. Comput. Programm.*, vol. 97, pp. 333–348, 2015.

[28] C. Xinyu, Y. Huiqun, and X. Xin, "Verification of hybrid Chi model for cyber-physical systems using PHAVer," in *Proc. 7th Int. Conf. Innovative Mobile Int. Serv. Ubiquitous Comput.*, 2013, pp. 122–128.

[29] G. Frehse, "Language overview for PHAVer version 0.35," 2006.

[30] J. Rushby, "Runtime certification," in *Proc. Int. Workshop Runtime Verif.*, 2008, pp. 21–35.

[31] S. Luna, A. Lopes, H. Y. S. Tao, F. Zapata, and R. Pineda, "Integration, Verification, Validation, Test, and Evaluation (IVVT&E) framework for System of Systems (SoS)," *Procedia Comput. Sci.*, vol. 20, pp. 298–305, 2013.

[32] F. Zapata, A. Akundi, R. Pineda, and E. Smith, "Basis path analysis for testing complex system of systems," *Procedia Comput. Sci.*, vol. 20, pp. 256–261, 2013.

[33] G. O. Langford, "Verification of requirements: System of systems theory, framework, formalisms, validity," in *Proc. INCOSE Int. Symp.*, 2017, pp. 780–795.

[34] S. Leśniewski, *S. Leśniewskis Lecture Notes in Logic*, J. J. T. Srzednicki, and Z. Stachniak, Eds. New York, NY, USA: Springer, 2012, vol. 24.

[35] D. Seo *et al.*, "Modeling and verification for different types of system of systems using PRISM," in *Proc. IEEE/ACM 4th Int. Workshop Softw. Eng. Syst. Syst.*, 2016, pp. 12–18.

[36] R. Calinescu and M. Z. Kwiatkowska, "Software engineering techniques for the development of Systems of Systems," in *Proc. Monterey Workshop*, 2008, pp. 59–82.

[37] G. Zhou and H. Zhao, "Modeling and quantitative safety analysis of Chinese train control system of systems," in *Proc. IEEE 18th Int. Conf. Intell. Transp. Syst.*, 2015, pp. 381–386.

[38] J. Bryans, J. Fitzgerald, R. J. Payne, and K. Kristensen, "Maintaining emergence in systems of systems integration: A contractual approach using SysML," Univ. Newcastle Upon Tyne, Newcastle upon Tyne, U.K., 2014.

[39] G. Brat, E. Denney, D. Giannakopoulou, J. Frank, and A. Jonsson, "Verification of autonomous systems for space applications," in *Proc. IEEE Aerosp. Conf.*, 2006, pp. 1–11.

[40] P. F. Fischer, D. Ludtke, V. Schaus, and A. Gerndt, "A formal method for early spacecraft design verification," in *Proc. IEEE Aerosp.*, 2013, pp. 1–8.

[41] P. Fischer, D. Ludtke, V. Schaus, O. Maibaum, and A. Gerndt, "Formal verification in early mission planning," in *Proc. SESP : Simul. EGSE Facil.*, 2012, pp. 1–8.

[42] P. Nuzzo *et al.*, "A contract-based methodology for aircraft electric power system design," *IEEE Access*, vol. 2, pp. 1–25, 2014.

[43] M. Rao, S. Ramakrishnan, and C. Dagli, "Modeling and simulation of net centric system of systems using systems modeling language and colored Petri-nets: A demonstration using the global earth observation system of systems," *Syst. Eng.*, vol. 11, no. 3, pp. 203–220, 2008.

[44] D. Drusinsky, J. B. Michael, and M. Shing, "Behavioral modeling and run-time verification of system-of-systems architectural requirements," in *Proc. Int. Conf. Computer, Commnun. Control Technol.*, vol. VI, 2004, pp. 13–18.

[45] T. Wongpiromsarn, U. Topcu, and R. M. Murray, "Formal synthesis of embedded control software for vehicle management systems," in *Proc. AIAA Infotech. Aerosp. Conf.*, 2011, pp. 1–17.

[46] P. Nilsson, N. Özay, U. Topcu, and R. M. Murray, "Temporal logic control of switched affine systems with an application in fuel balancing," in *Proc. Amer. Control Conf.*, 2012, pp. 5302–5309.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[47] O. Mickelin, N. Özay, and R. M. Murray, "Synthesis of correct-by-construction control protocols for hybrid systems using partial state information," in *Proc. Amer. Control Conf.*, 2014, pp. 2305–2311.
[48] "ECSS-E-ST-40C: Space Engineering—Software." European Cooperation for Space Standardization, Zurich, The Netherlands, 2009.
[49] X. Gan, J. Dubrovin, and K. Heljanko, "A symbolic model checking approach to verifying satellite onboard software," *Sci. Comput. Programm.*, vol. 82, pp. 44–55, 2014.

Vittoria Nardone received the M.Sc. degree in computer engineering from the University of Sannio, Benevento, Italy, where she is currently working toward the Ph.D. degree under the supervision of Antonella Santone.

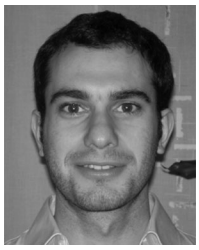
She applies the following technique on the Android malware analysis environment in order to identify malicious behaviors. Her current research interests include application on formal verification methods, in particular on the model checking technique.

Antonella Santone received the Laurea degree in computer science from the University of Pisa, Pisa, Italy, in 1993 and 1997, respectively and the Ph.D. degree in computer systems engineering from the Department of Information Engineering, University of Pisa, in 1997.

Since September 2017, she has been an Associate Professor with the University of Molise, Campobasso, Italy. From November 1998 to October 2001, she was an Assistant Professor with the University of Pisa. From November 2001 to August 2017, she was an Associate Professor with the Department of Engineering, University of Sannio. Her research interests include formal description languages, temporal logic, concurrent and distributed systems modeling, heuristic search, and formal methods for systems biology and for software security.

Massimo Tipaldi received the master's degree in computer science engineering (with a specialization in automation and control systems) from the University of Sannio, Benevento, Italy, in 1988, and the Ph.D. degree in information technology with a dissertation focusing on model-based autonomy for space systems.

He is a Space Software Project Manager/System Engineer with OHB Italia Spa/OHB System AG and possesses about 20 years of experience in the managerial and technical coordination of ESA/ASI/CNES SW projects (satellite systems, experimental equipment for the International Space Station, and ground segments). He holds 1 patent and has co-authored more than 25 papers published in proceedings of international conferences or international archival journals. His research interests include critical SW systems, reinforcement/machine learning techniques, formal verification, and advanced system control techniques.



Davide Liuzza received the Ph.D. degree in automation engineering from the University of Naples Federico II, Naples, Italy, in April 2013.

He was a Visiting Ph.D. Student with the Department of Applied Mathematics, University of Bristol, Bristol, U.K., from January 2012 to February 2012, and with the ACCESS Linnaeus Centre, Royal Institute of Technology (KTH), Stockholm, Sweden, from July 2012 to January 2013. From September 2013 to September 2015, he was a Postdoctoral Scholar with the Automatic Control Laboratory, KTH. He is currently a Postdoctoral Researcher with the Department of Engineering, University of Sannio, Benevento, Italy. His research interests include networked control systems, coordination of multiagent systems, incremental stability of nonlinear systems, nonlinear control and hybrid systems, and analysis and control of systems of systems.

Dr. Liuzza was the recipient of a postdoctoral scholarship.

Luigi Glielmo was born in 1960. He received the Laurea degree in electronic engineering.

He is currently a Research Doctorate of automatic control. He taught at the University of Palermo, University of Naples Federico II, and University of Sannio, Benevento, Italy, where he is now a Professor of automatic control. He has co-authored more than 130 papers in international archival journals or proceedings of international conferences, co-edited 2 books, and holds 3 patents. From 2001 to 2007, he was the Head of the Department of Engineering, University of Sannio, where he is currently Rector's Delegate for Technology Transfer and Coordinator of the Ph.D. course on information technologies for engineering. His research interests include singular perturbation methods, model predictive control methods, automotive controls, deep brain stimulation modeling and control, and smart-grid control.

Dr. Glielmo is a member of Editorial Boards of archival journals. He has been the Chair of the IEEE Control Systems Technology Committee on Automotive Controls.